Week 11: Agile Methods in Research and Development

R&D and Project Management in Computer Science

R&D and Project Management in Computer Science (11117BLG002). Prof. Dr. Utku Kose. Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye. Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India). ORCID 0000-0002-9652-6415. E-mail: utkukose@sdu.edu.tr, utku.kose@und.edu, ukose@up.edu.mx, utkukose@gmail.com. This course is updated in line with current developments in the field. First version: September 2026. Last update: October 2026.

Overview

Plans made at the start of a research project rarely survive contact with results. This week introduces the values of agile development, the Scrum framework, Kanban and the law that links work in progress to cycle time. It also discusses how iterative methods can work inside grant agreements with fixed deliverables [1, 2, 3].

Learning outcomes

By the end of the week, students are expected to explain the values and principles of agile development and to describe the accountabilities, events and artefacts of Scrum. They should be able to apply Little's law to a team's flow of work and to forecast delivery from past velocity. Finally, they should be able to design a hybrid approach for a funded research project.

Lecture

Agile values

In 2001, a group of software practitioners published the Manifesto for Agile Software Development [1]. It values individuals and interactions over processes and tools, and working software over comprehensive documentation. It also values customer collaboration over contract negotiation, and responding to change over following a plan. It still recognises value in the items on the right. Twelve principles elaborate these values, among them the frequent delivery of working software and regular reflection on how to become more effective. Dingsøyr and colleagues reviewed a decade of research on agile methods and argued that the field needed stronger theoretical foundations to explain why and when the methods work [4].

Scrum

The Scrum Guide describes Scrum as a lightweight framework for addressing complex problems [2]. A Scrum Team has three accountabilities. Developers create the increment, and a

Product Owner maximises the value of the product and manages the Product Backlog. A Scrum Master helps the team use Scrum effectively. Work proceeds in Sprints of one month or less, each with Sprint Planning, a Daily Scrum, a Sprint Review and a Sprint Retrospective. The three artefacts, the Product Backlog, the Sprint Backlog and the Increment, carry commitments: the Product Goal, the Sprint Goal and the Definition of Done. The velocity of past Sprints supports forecasts, which the interactive lab turns into probability distributions.

Flow and Little's law

Kanban, as described by Anderson, makes work visible on a board, limits the work in progress and manages the flow of items through the system [5]. Little proved that, in a stable system, the average number of items in the system equals the arrival rate times the average time an item spends in it [3]. For a team, this means that the average cycle time equals the average work in progress divided by the throughput. If throughput is limited by the team's capacity, starting more work does not finish more work; it only makes each item take longer, as Figure 11.1 shows. Researchers who work on many papers and tasks at once experience the same effect.

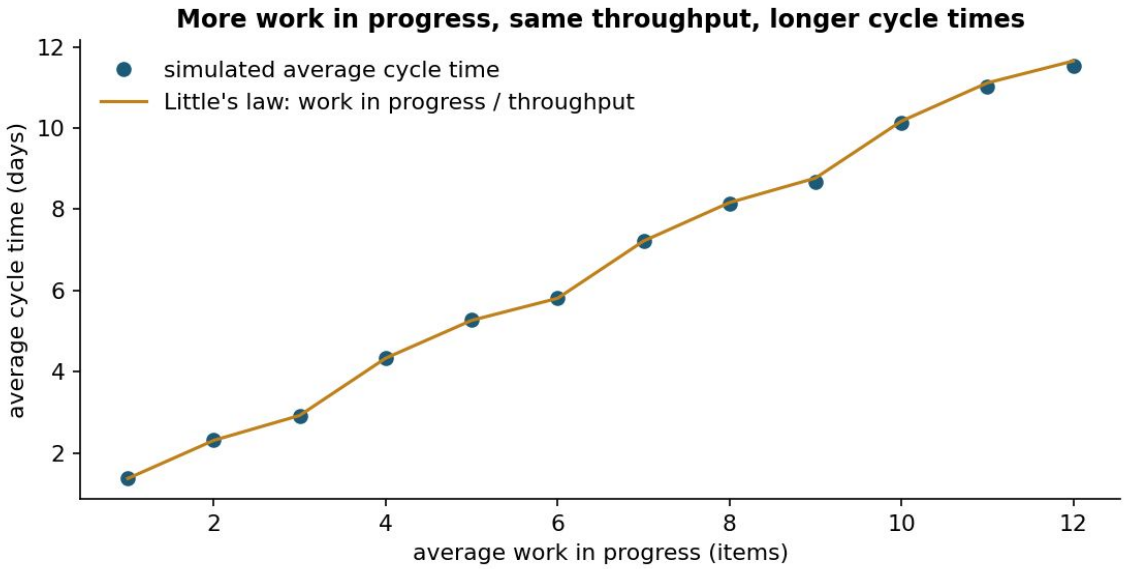


Figure 11.1. Simulated Kanban system with fixed capacity: raising the limit on work in progress leaves throughput unchanged and lengthens cycle times, as Little's law predicts [3].

Agile inside a research grant

Research projects combine a fixed structure, with work packages, deliverables and milestones agreed with the funder, and uncertain content, since results determine the next steps. Marchesi and colleagues described the use of a distributed form of Scrum to manage a European research project that developed an agent-based software platform [6]. They argued that research projects are complex and must be adapted continuously. A common hybrid keeps the work packages and milestones of the grant as the stable frame and runs iterations inside them. The iterations use a backlog of hypotheses, experiments and software tasks, short cycles with reviews against the

objectives, and limits on work in progress. Changes that affect deliverables still follow the funder's amendment rules.

> ***Pause and reflect.*** *How many research tasks do you have in progress now? What does Little's law predict about their completion?*

Hands-on components

Interactive lab: Part A simulates a team whose capacity is shared among the items in progress and checks Little's law. Part B forecasts the number of Sprints needed for a backlog from past velocities. Part C sorts the elements of Scrum [2, 3]. Link: https://utkukose.github.io/rd-project-management-NB-lecture/weeks/week-11/lab.html

Colab notebook: It simulates a Kanban system to check Little's law and forecasts the number of Sprints needed for a backlog by resampling past velocities [2, 3]. Link: https://colab.research.google.com/github/utkukose/rd-project-management-NB-lecture/blob/main/weeks/week-11/NB11_agile_research.ipynb

Reflection prompts

1. Which of your current tasks would you stop or pause to reduce your work in progress, and what would you expect to happen?
2. What would a Definition of Done look like for an experiment or a paper in your group?
3. Where does a grant agreement constrain agile work most in your experience, and how could the frame be designed to leave room for learning?

Weekly task and submission

What to do. Design the working method of your project. Describe five elements: the stable frame that the grant fixes, the backlog, the length of an iteration, the limit on work in progress and the Definition of Done for experiments, software and papers. Then forecast how many iterations one work package needs, based on past or assumed velocities.

Where to look. Lecture: sections "Scrum", "Flow and Little's law" and "Agile inside a research grant". Interactive lab: Part A for work in progress and cycle time, and Part B for the forecast from past velocity. Colab notebook: Section 2 with Exercise 2, which forecasts the number of Sprints by resampling past velocities.

Example. A Definition of Done for an experiment could require version control for the code, fixed random seeds and results logged with the version of the data. A backlog of 60 points with past velocities of 8 to 12 points needs about 6 iterations on average, but a forecast at the 85th percentile may need 7.

What to submit. A description of about one page with the forecast, and the Colab notebook with Exercises 1 and 2 completed.

The weekly task supports self-learning and builds a personal portfolio. In an active semester, students who follow the course with the instructor can send their work for evaluation to utkukose@sdu.edu.tr or utkukose@gmail.com. The e-mail contains the items that the task lists under What to submit. It also includes the learning log exported from the interactive lab (tab: Reflect and submit).

References

[1] Beck, K., Beedle, M., van Bennekum, A., et al. (2001). Manifesto for Agile Software Development. https://agilemanifesto.org

[2] Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. https://scrumguides.org

[3] Little, J. D. C. (1961). A proof for the queuing formula: L = λW. Operations Research, 9(3), 383-387. https://doi.org/10.1287/opre.9.3.383

[4] Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. B. (2012). A decade of agile methodologies: Towards explaining agile software development. Journal of Systems and Software, 85(6), 1213-1221. https://doi.org/10.1016/j.jss.2012.02.033

[5] Anderson, D. J. (2010). Kanban: Successful Evolutionary Change for Your Technology Business. Blue Hole Press.

[6] Marchesi, M., Mannaro, K., Uras, S., & Locci, M. (2007). Distributed Scrum in research project management. In Agile Processes in Software Engineering and Extreme Programming (XP 2007), Lecture Notes in Computer Science 4536 (pp. 240-244). https://doi.org/10.1007/978-3-540-73101-6_45